

Milestone-5 Report

An Ensemble Approach to Multiple-Choice Question Answering: DeBERTa and RoBERTa

Kaggle: Smart MCQ Solver Challenge

NAINI DIWAN
Date: 21 July 2026

1. Abstract

A machine learning pipeline was built to solve a five-option multiple-choice question (MCQ) answering task, the “Smart MCQ Solver Challenge” on Kaggle, hosted by IIT Madras. First we develop three independent models of increasing sophistication: a TF-IDF cosine-similarity baseline, a fine-tuned DeBERTa-v3-small transformer, and a LoRA-adapted RoBERTa-base transformer. Then we combined the two transformer models into a weighted ensemble for final prediction.

The dataset consists of 2,000 labeled training examples, each with a prompt, five answer options (A–E), and a single correct answer. Performance is measured using accuracy, macro F1, and Mean Average Precision at 3 (MAP@3). Both transformer models achieved validation accuracy above 96%. The final weighted ensemble (70% DeBERTa, 30% RoBERTa) reached a validation accuracy of 97.50%, a macro F1 of 0.9746, and a MAP@3 of 0.9833, the strongest result of the four approaches tested.

2. Introduction

A supervised classification problem was targeted: given a question prompt and five candidate answers labeled A through E, predict which single option is correct. Via MAP@3, the pipeline is designed to output a ranked list of the top three most likely answers per question instead of a single label.

Training data (train.csv) contains 2,000 rows with columns id, prompt, A, B, C, D, E, and answer. A held-out test.csv, of the same structure minus the answer column, is used for final inference. All splits use an 80/20 stratified train/validation split (random_state=42) on the answer column. A fixed global seed (42) is set across the workflow for reproducibility, though full bit-for-bit determinism across GPU runs is not guaranteed by these settings alone.

3. Methodology

3.1 Evaluation Metric

calculate_metrics function is shared across all three models. It receives logits and true labels, & computes:

- Accuracy : proportion of top-1 predictions matching the true label
- Macro F1 : F1 score averaged equally across the five classes, unweighted by class frequency
- MAP@3 : for each example, 1/rank is awarded if the true label appears in the top-3 ranked predictions (1.0 if rank 1, 0.5 if rank 2, 0.333 if rank 3, 0 otherwise), averaged over all examples

For MAP@3, argsort is required instead of the max operation because it needs the full ranking of options, not only the single highest-probability answer.

3.2 Model 1 — TF-IDF Cosine Similarity

The baseline model requires no training. A TfidfVectorizer (max_df=0.95, discarding terms in over 95% of documents) is fit on the combined vocabulary of all training prompts and all five answer options. For each validation example, the prompt and its five options are each transformed into TF-IDF vectors, and cosine

similarity is computed between the prompt vector and each option vector. The five similarity scores are treated as unnormalized “probabilities” and ranked to produce a MAP@3 prediction.

TF-IDF down-weights uninformative high-frequency terms relative to a raw count vectorizer, and cosine similarity measures directional similarity independent of document length, which suits comparing a long prompt against short answer options. But it is unaware of the semantic information in the data.

3.3 Model 2 — Fine-Tuned DeBERTa-v3-small

The second model fine-tunes microsoft/deberta-v3-small end-to-end using Hugging Face's AutoModelForMultipleChoice head, which supports the (prompt, option) pairing structure. DeBERTa's disentangled attention mechanism, which represents each token's content and relative position as separate vectors, is the reason for choosing such architecture over a standard BERT-style encoder.

HuggingFaceMCQDataset class tokenizes each (prompt, option) pair for all 5 options into a rectangular tensor of shape [5, max_length], padding and truncating to max_len=128. This length was chosen empirically: token-length percentiles computed over the training prompts showed a 99th percentile of approximately 45 tokens, so 128 provides comfortable headroom without excessive padding. A matching mcq_data_collator stacks these per-example tensors into batches for the Hugging Face Trainer.

Training uses 4 epochs, a learning rate of 1e-5 with cosine decay and a 10% warmup ratio, gradient clipping at a max norm of 0.5, and gradient accumulation of 2 steps on a per-device batch size of 8 (effective batch size 32). The best checkpoint (by validation metric) is retained via load_best_model_at_end, and training/evaluation metrics are logged to Weights & Biases.

3.4 Model 3 — LoRA-Tuned RoBERTa-base

The third model applies Low-Rank Adaptation (LoRA) to roberta-base. LoraConfig sets rank r=8, lora_alpha=16, a dropout of 0.1, no bias adaptation, and targets the query and value projection matrices within the attention layers, following the adaptation pattern from the original LoRA paper. This freezes the pretrained RoBERTa weights and trains only a small number of injected rank-decomposition matrices, substantially reducing trainable parameters and memory footprint relative to full fine-tuning.

Training runs for 10 epochs with a higher learning rate (5e-4) than DeBERTa run, since only the small adapter matrices are updated and can tolerate larger steps. Logits are passed through a softmax to obtain per-option probabilities. Sigmoid isn't utilized as this is single-label multi-class classification problem.

3.5 Model 4 — Weighted Ensemble

Finally, the ensemble is run over the validation set, converting each model's logits to softmax probabilities. Then probabilities are combined via linear weighting: $0.70 \times \text{DeBERTa probability} + 0.30 \times \text{RoBERTa probability}$. Same weighting is then applied to test set for the final submission, and the top three ranked classes per example are written to submission.csv in the space-separated format (e.g. “A B D”).

4. Results

Validation-set performance for all four modeling stages is summarized below.

Model	Accuracy	Macro F1	MAP@3
1. TF-IDF (cosine similarity)	0.125	0.123	0.271
2. DeBERTa-v3-small (fine-tuned)	0.205	0.175	0.349
3. RoBERTa-base + LoRA	0.975	0.974	0.983
4. Ensemble (0.7 DeBERTa + 0.3 RoBERTa)	0.975	0.975	0.983

The gap between the TF-IDF baseline and the transformer models is large: MAP@3 rises from 0.2712 to 0.9833, a roughly 3.6× improvement. This is because TF-IDF is not context-aware. It treats text as a bag of words, ignoring word order, grammar, and local context. Benefit of ensembling: the individual models make partially independent errors, and averaging their probability distributions smooths out some of those individual mistakes.

Five example predictions from the final submission file are shown below to illustrate the output format (each row lists the top-3 predicted options in descending confidence order).

id	Top-3 Prediction
1	A B D
2	B E D
3	B E D
4	E C D
5	C A D

5. Discussion

Why the Baseline Underperforms

A validation accuracy of 12.5% against five equally likely options (chance = 20%) shows the TF-IDF model performing worse than random guessing on this data, possibly due to data leakage. A lexical-similarity method is poorly suited to a knowledge-based MCQ task, because correct answers might be phrased in different terminology from the question. This would push a cosine-similarity towards wrong option.

Ensemble Weighting

The 70/30 weighting favors DeBERTa, due to its disentangled attention mechanism and it was full-parameter fine-tuned.

6. Limitations

Data Leakage in TF-IDF Preprocessing

While TF-IDF preprocessing, the test set's prompt column was mistakenly lowercased using the training DataFrame rather than the test DataFrame itself:

```
test_df['prompt'] = df['prompt'].apply(lambda x: x.lower())
```

Unjustified Ensemble Weights

0.7/0.3 split is not derived from a validation sweep in the visible cells. A brief grid search, or a validation-set logistic regression/optimization over the two probability vectors, would substantiate the choice.

Confounded Comparison Between Model 2 and Model 3

Model 2 (DeBERTa) and Model 3 (RoBERTa) differ simultaneously in architecture, fine-tuning strategy (full vs. LoRA), epoch count, and learning rate. This makes it impossible, to attribute either model's performance to any single one of these factors. Running a LoRA-adapted DeBERTa and/or a fully fine-tuned RoBERTa as additional ablations would isolate the effect of the adaptation method from the effect of the base architecture.

7. Conclusion

A coherent, well-commented pipeline was implemented from a weak lexical baseline to two independently trained transformer classifiers and a weighted probability ensemble. The final ensemble model achieves strong validation performance (97.50% accuracy, 0.9833 MAP@3), a substantial improvement over the TF-IDF baseline (12.50% accuracy, 0.2712 MAP@3).

The main outstanding issues are the confirmed test-set lowercasing bug in the TF-IDF stage, the absence of standalone validation metrics for the two transformer models prior to ensembling, and the unvalidated 70/30 ensemble weighting. None of these compromise the final ensemble score, which is computed correctly from val_df, but addressing them would let the results be verified with more confidence.